

REHOBOTH IMPRESSION

Une marque de REHOBOTH MULTISERVICES

GUIDE DE DÉVELOPPEMENT

Phase 1 — Méthodologie, stack technique et sécurité pour l'équipe de développement

Porteur de projet : Roland – REHOBOTH MULTISERVICES

Localisation : Abidjan, Côte d'Ivoire

Durée indicative : 6 semaines (1,5 mois)

Version 1.0 – Juin 2026

Ce document s'adresse à la ou aux personnes qui développent la plateforme. Il complète le « Cahier des charges – Phase 1 », qui reste la référence pour les fonctionnalités, les règles de tarification et le cadre légal. Ce document-ci répond à une autre question : comment construire concrètement la Phase 1, avec quels outils, selon quelle méthode, et dans quelles conditions de sécurité.

Table des matières

Table des matières	2
1. Objectif de ce document	3
2. Choix technologiques recommandés	3
2.1 Langages et frameworks	3
2.2 Hébergement	4
2.3 Paiement en ligne	4
3. Protocole de sécurité pour le développement	4
3.1 Authentification et accès	5
3.2 Protection des fichiers et données clients	5
3.3 Validation des entrées et des fichiers	5
3.4 Paiement	5
3.5 Infrastructure et bonnes pratiques générales	5
4. Exigences d'ergonomie et d'expérience utilisateur (UX)	6
4.1 Principes de conception mobile-first	6
4.2 Adaptation à l'ordinateur	6
4.3 Validation de l'ergonomie	6
5. Méthodologie de développement — feuille de route sur 6 semaines	7
5.1 Semaine 1 — Fondations techniques	7
5.2 Semaine 2 — Dépôt et analyse du document (F1, F2)	7
5.3 Semaine 3 — Devis, reliure et parcours de commande (F3, F4, F5)	7
5.4 Semaine 4 — Paiement en ligne (F6)	8
5.5 Semaine 5 — Tableau de bord opérateur, notifications et pages légales (F7, F8, F9, F10)	8
5.6 Semaine 6 — Tests, sécurité, conformité et lancement	8
6. Le rôle de l'assistant IA pendant le développement	8
7. Annexe — Checklists récapitulatives	9
7.1 Checklist sécurité (à vérifier avant le lancement, semaine 6)	9
7.2 Variables d'environnement et configuration à prévoir	9

1. Objectif de ce document

Le « Cahier des charges – Phase 1 » décrit CE QUE la plateforme doit faire (fonctionnalités, tarifs, règles de gestion, cadre légal). Ce guide décrit COMMENT la construire : avec quelles technologies, selon quel calendrier, avec quelles précautions de sécurité, et avec quel niveau d'exigence sur l'ergonomie pour les utilisateurs finaux.

- Une stack technique recommandée, choisie pour aller vite sans sacrifier la fiabilité — avec les raisons de ce choix, pas seulement la décision elle-même.
- Un hébergement recommandé pour chaque composant du système.
- Un protocole de sécurité à respecter dès le premier jour de développement, pas seulement à la fin.
- Des exigences d'ergonomie claires, pensées mobile en premier, pour des utilisateurs ivoiriens qui navigueront majoritairement depuis un smartphone, parfois avec une connexion lente.
- Une feuille de route semaine par semaine sur 6 semaines, avec un jalon de validation concret à chaque étape.

Claude (cet assistant) accompagne le développement tout au long de ces 6 semaines — voir § 6 pour le détail de ce rôle.

2. Choix technologiques recommandés

2.1 Langages et frameworks

Pour aller vite sur 6 semaines, le critère le plus important n'est pas « quel est le langage le plus puissant », mais « quel est le langage qui réduit le temps de développement et le risque d'échec sur les parties les plus délicates du projet ». Ici, la partie la plus délicate est sans conteste le moteur de classification N&B / Couleur (§ F2 du cahier des charges Phase 1) : il faut transformer chaque page d'un PDF en image et analyser sa distribution de couleurs.

Composant	Choix recommandé	Pourquoi
Frontend	Next.js (React + TypeScript)	Écosystème mature, composants réutilisables, bon support du rendu mobile-first, et déjà utilisé sur vos précédents projets — pas de courbe d'apprentissage supplémentaire.
Backend	Python + FastAPI	Les bibliothèques Python de traitement de PDF et d'image (pypdf, pdf2image, Pillow) sont plus matures et plus simples à mettre en œuvre que leurs équivalents Node.js pour la classification couleur — c'est précisément la partie la plus risquée du projet. FastAPI est rapide à écrire et génère une documentation d'API automatique, utile pour le suivi du projet.
Base de données	PostgreSQL (hébergé sur Neon)	Robuste, gratuit pour démarrer, déjà utilisé sur un précédent projet.
Stockage des fichiers PDF	Cloudflare R2	Coût très faible, compatible avec l'API S3 (standard largement documenté), permet de configurer une règle de suppression

Composant	Choix recommandé	Pourquoi
		automatique des fichiers après un délai (cf. § 4.2 du cahier des charges Phase 1).

Alternative plus légère : si vous préférez limiter le nombre de langages à un seul, un Next.js « fullstack » (frontend + API routes, sans backend Python séparé) est possible et réduit le nombre de services à maintenir. Le compromis : la classification couleur sera plus longue à développer et moins fiable au départ avec les bibliothèques Node.js disponibles. La recommandation Python + FastAPI reste préférable pour respecter le délai de 6 semaines, sauf si l'équipe de développement est déjà beaucoup plus à l'aise en Node.js qu'en Python.

2.2 Hébergement

Composant	Hébergeur recommandé	Pourquoi
Frontend (Next.js)	Vercel	Support natif de Next.js sans configuration complexe, déploiement automatique à chaque mise à jour du code, plan gratuit largement suffisant pour démarrer.
Backend (FastAPI)	Railway	Déploiement simple d'une API Python, déjà utilisé sur un précédent projet, bonne intégration avec PostgreSQL.
Base de données	Neon (PostgreSQL serverless)	Déjà utilisé, mise à l'échelle automatique, plan gratuit généreux pour démarrer une activité naissante.
Stockage des fichiers PDF	Cloudflare R2	Coût très faible, et seul composant Cloudflare utilisé ici — voir remarque ci-dessous.

Point d'attention : ne pas héberger le frontend Next.js directement sur Cloudflare Pages pour cette Phase 1. L'expérience déjà vécue sur un précédent projet (échecs de build avec l'adaptateur @cloudflare/next-on-pages) montre que cette voie consomme du temps de débogage qui n'est pas disponible sur un délai de 6 semaines. Garder Cloudflare uniquement pour le stockage R2, et héberger le frontend sur Vercel, qui est conçu nativement pour Next.js.

2.3 Paiement en ligne

Pour intégrer Wave et Orange Money rapidement sans gérer deux intégrations séparées, utiliser un agrégateur de paiement local (par exemple CinetPay ou PayDunya) qui expose les deux moyens de paiement derrière une seule API. L'intégration directe de chaque API (Wave Business, Orange Money) reste possible plus tard, une fois le volume de commandes suffisant pour justifier la réduction de frais de transaction que cela permettrait.

Règle de sécurité non négociable : ne jamais marquer une commande comme « payée » sur la seule base d'une redirection côté client après paiement. Toujours attendre la confirmation serveur-à-serveur (webhook) envoyée par l'agrégateur, avec vérification de la signature du webhook, avant d'autoriser le passage en file d'impression.

3. Protocole de sécurité pour le développement

Cette plateforme traite des documents potentiellement sensibles (relevés bancaires, dossiers d'entreprise, copies d'identité) et de l'argent réel via Wave/Orange Money. La sécurité n'est pas

une étape finale avant le lancement : elle doit être appliquée dès les premières lignes de code. La liste ci-dessous est la base à respecter pendant tout le développement.

3.1 Authentification et accès

- L'accès au tableau de bord opérateur/administrateur doit être vérifié côté serveur (middleware), jamais uniquement côté client (un simple « if » dans le code du navigateur ne protège rien — une faille de ce type a déjà été identifiée et corrigée sur un précédent projet).
- Mots de passe des comptes opérateurs hachés (jamais stockés en clair), avec une bibliothèque standard reconnue (ex. bcrypt ou argon2).
- Sessions ou jetons d'authentification (JWT) avec une durée de vie limitée, transmis via cookies sécurisés (httpOnly, secure, sameSite) plutôt que stockés en clair dans le navigateur.

3.2 Protection des fichiers et données clients

- Les fichiers PDF stockés sur Cloudflare R2 doivent être dans un bucket privé, jamais listable ou accessible publiquement : l'accès se fait uniquement via des liens signés à durée limitée, générés par le backend.
- Suppression automatique des documents après le délai défini au § 4.2 du cahier des charges Phase 1, mise en œuvre via une règle de cycle de vie (lifecycle) sur le bucket, et non comme une tâche manuelle à ne pas oublier.
- Aucune donnée sensible (contenu du document, numéro de carte, etc.) ne doit jamais apparaître dans les journaux d'application (logs) ou les messages d'erreur.

3.3 Validation des entrées et des fichiers

- Toute validation effectuée côté client (type de fichier, taille) doit être systématiquement revérifiée côté serveur : un client malveillant peut contourner n'importe quelle vérification côté navigateur.
- Limiter strictement les types de fichiers acceptés au format PDF (vérification du contenu réel du fichier, pas seulement de son extension).
- Valider toutes les entrées utilisateur (formulaires, paramètres d'API) avec un outil de validation systématique (ex. les modèles Pydantic de FastAPI), pour éviter les injections et les données malformées.
- Utiliser un ORM (ex. SQLAlchemy) avec des requêtes paramétrées plutôt que des requêtes SQL construites manuellement, pour éliminer le risque d'injection SQL.

3.4 Paiement

- Vérifier systématiquement la signature de chaque webhook reçu de l'agrégateur de paiement avant de traiter l'événement, pour éviter qu'un tiers ne puisse simuler une fausse confirmation de paiement.
- Ne jamais stocker de données de carte bancaire ou d'identifiants de paiement directement sur les serveurs du projet : ces informations restent gérées par l'agrégateur, qui est conçu pour cela.

3.5 Infrastructure et bonnes pratiques générales

- HTTPS obligatoire sur l'ensemble du site (géré automatiquement par Vercel et Railway, à vérifier dès la mise en ligne).
- Toutes les clés secrètes (API de paiement, base de données, etc.) stockées en variables d'environnement, jamais codées en dur dans le code ni commises dans le dépôt Git.

- Limitation du débit (rate limiting) sur les routes sensibles (dépôt de fichier, tentative de connexion) pour limiter les risques d'abus ou de saturation du service.
- Configuration stricte du CORS pour n'autoriser que le domaine officiel du site à appeler l'API backend.
- Vérification régulière des dépendances du projet (ex. `npm audit`, `pip-audit`) pour repérer les bibliothèques présentant des failles connues.

Ce protocole doit être revu une dernière fois pendant la semaine 6 (§ 5.6) avant le lancement commercial, mais il s'applique dès la semaine 1.

4. Exigences d'ergonomie et d'expérience utilisateur (UX)

Le site doit être simple à comprendre et à utiliser, en priorité sur mobile, sans sacrifier le confort sur ordinateur. La majorité des utilisateurs ivoiriens navigueront depuis un smartphone, parfois avec une connexion lente — c'est cette situation qui doit guider les choix de conception, pas le confort d'un grand écran de bureau.

4.1 Principes de conception mobile-first

- Concevoir d'abord l'interface pour un écran de smartphone, puis l'adapter à un écran d'ordinateur — jamais l'inverse.
- Un parcours de commande court et linéaire : dépôt du PDF → devis → choix d'options → coordonnées → paiement, sans étape inutile ni écran de confirmation superflu.
- Des zones cliquables suffisamment grandes pour un usage au doigt (boutons, cases à cocher), sans avoir besoin de zoomer.
- Un langage simple, en français courant, sans jargon technique (ex. préférer « Votre document est prêt » à un statut technique comme « PROCESSED »).
- Un indicateur de progression visible pendant les étapes de la commande, pour que le client sache toujours où il en est.
- Des temps de chargement optimisés pour une connexion mobile moyenne ou faible : images compressées, code minimal chargé à chaque étape, retour visuel immédiat pendant l'analyse du PDF (qui peut prendre quelques secondes) plutôt qu'un écran figé sans explication.

4.2 Adaptation à l'ordinateur

- Sur un écran plus large, exploiter l'espace disponible pour afficher davantage d'informations en une seule vue (ex. devis détaillé et aperçu page par page côte à côte), sans simplement étirer la version mobile.
- Le tableau de bord opérateur (§ F9 du cahier des charges Phase 1), utilisé principalement sur ordinateur, peut privilégier la densité d'information (listes, filtres, tableaux) plutôt que la simplicité extrême attendue côté client.

4.3 Validation de l'ergonomie

- Tester le parcours client sur un véritable smartphone d'entrée ou moyenne gamme (pas uniquement sur l'écran large d'un ordinateur de développement), à un moment donné de chaque semaine de développement plutôt qu'une seule fois à la fin.
- Faire tester le parcours par une personne extérieure au projet avant le lancement (§ 5.6), pour repérer les points de confusion qu'un développeur habitué au site ne remarque plus.

5. Méthodologie de développement — feuille de route sur 6 semaines

La feuille de route suit l'ordre logique des fonctionnalités du cahier des charges Phase 1 (F1 à F10) : on construit d'abord les briques les plus risquées techniquement (l'analyse du PDF), puis on enchaîne vers le devis, le paiement, le tableau de bord, et enfin les tests et le lancement. Chaque semaine se termine par un jalon concret et vérifiable, plutôt qu'une simple liste de tâches cochées.

Démarche en parallèle dès la semaine 1 : la déclaration auprès de l'Autorité de Protection des Données à Caractère Personnel (§ 6.2 du cahier des charges Phase 1) doit être engagée dès le début du projet, en parallèle du développement, car le délai administratif de traitement est imprévisible et peut dépasser la durée du développement lui-même. Ne pas attendre la semaine 6 pour s'en occuper.

5.1 Semaine 1 — Fondations techniques

- Mise en place des dépôts de code (frontend et backend), structure de projet, premières règles de qualité de code.
- Création du schéma initial de base de données sur Neon (commandes, statuts, comptes opérateurs).
- Configuration du bucket Cloudflare R2 avec règle de suppression automatique des fichiers.
- Premier déploiement, même minimal, sur Vercel (frontend) et Railway (backend), pour valider toute la chaîne technique dès le départ plutôt qu'à la fin du projet.
- Maquettes (wireframes) mobile-first du parcours client, validées avec Roland avant de coder l'interface définitive.

Jalon de la semaine 1 : une page d'accueil minimale est en ligne, accessible par une adresse publique, et communique avec la base de données et le stockage de fichiers — la preuve que toute la chaîne technique fonctionne de bout en bout.

5.2 Semaine 2 — Dépôt et analyse du document (F1, F2)

- Interface de dépôt de fichier PDF (glisser-déposer et sélection classique), pensée mobile en premier.
- Validation côté serveur : format PDF uniquement, taille maximale, détection des fichiers corrompus ou protégés par mot de passe.
- Extraction du nombre de pages du document.
- Construction du moteur de classification Noir & Blanc / Couleur page par page — la partie technique la plus délicate du projet, à traiter en priorité pour détecter tout problème de faisabilité le plus tôt possible.
- Affichage du détail page par page au client, conformément à la règle de transparence du § F2 du cahier des charges Phase 1.

Jalon de la semaine 2 : un vrai document PDF déposé par un testeur affiche un nombre correct de pages N&B et couleur, vérifié manuellement sur plusieurs types de documents (texte pur, document avec logo couleur, scan de qualité moyenne).

5.3 Semaine 3 — Devis, reliure et parcours de commande (F3, F4, F5)

- Moteur de calcul du devis selon la formule du § 3.3 du cahier des charges Phase 1.
- Affichage du devis détaillé et instantané au client.
- Choix de la reliure (case à cocher, tarif automatique selon la tranche de pages).

- Choix du mode de récupération (retrait en agence ou livraison) et saisie des coordonnées client.
- Mise en place des statuts de commande en base de données, conformément à la liste du § F7 du cahier des charges Phase 1.

Jalon de la semaine 3 : un parcours complet, du dépôt du document jusqu'à un récapitulatif de commande prêt à être payé, avec un calcul de prix exact vérifié sur plusieurs cas de test (incluant l'exemple chiffré du § 3.4 du cahier des charges Phase 1).

5.4 Semaine 4 — Paiement en ligne (F6)

- Intégration de l'agrégateur de paiement (Wave et Orange Money) en environnement de test (sandbox).
- Mise en place du webhook de confirmation de paiement avec vérification de signature.
- Génération automatique d'un reçu et d'un numéro de commande consultable par le client.
- Tests de paiement couvrant à la fois les scénarios de succès et d'échec.

Jalon de la semaine 4 : une commande de test passe du statut « devis » au statut « payée » uniquement après confirmation réelle reçue du prestataire de paiement, jamais sur la seule base d'une redirection côté client.

5.5 Semaine 5 — Tableau de bord opérateur, notifications et pages légales (F7, F8, F9, F10)

- Interface opérateur : liste des commandes filtrable par statut, changement de statut en un clic, accès au fichier à imprimer.
- Authentification sécurisée de l'espace opérateur/administrateur (vérification côté serveur, voir § 3.1).
- Mise en place des notifications client (WhatsApp en priorité, en complément du SMS et de l'email) aux étapes clés de la commande.
- Mini-tableau de bord de statistiques pour Roland (commandes, chiffre d'affaires, pages imprimées).
- Intégration des pages légales (politique de confidentialité, CGU/CGV, grille tarifaire publique), avec le contenu déjà rédigé dans le cahier des charges Phase 1 (§ 6).

Jalon de la semaine 5 : un opérateur peut traiter une commande de bout en bout depuis le tableau de bord, et les pages légales sont en ligne et accessibles avant tout paiement réel.

5.6 Semaine 6 — Tests, sécurité, conformité et lancement

- Recette complète sur la base des dix critères du § 9 du cahier des charges Phase 1.
- Revue de sécurité finale sur la base de la checklist du § 7.1 de ce document.
- Test d'ergonomie mobile sur des appareils réels d'entrée ou moyenne gamme, représentatifs des utilisateurs ivoiriens.
- Vérification de l'état de la déclaration auprès de l'Autorité de Protection des Données, engagée dès la semaine 1.
- Lancement progressif (soft launch) auprès des clients existants de REHOBOTH avant toute communication large, avec un suivi rapproché des premières commandes réelles.

Jalon de la semaine 6 : le site est en production, les premières commandes réelles sont traitées avec succès, et un suivi rapproché est en place pour réagir rapidement à tout problème observé dans les premiers jours.

6. Le rôle de l'assistant IA pendant le développement

Claude accompagne ce projet à chaque semaine de la feuille de route ci-dessus, en complément du travail de développement proprement dit :

- Aide à la conception technique : choix d'architecture, structure de base de données, découpage des tâches en début de semaine.
- Débogage des points bloquants, en particulier sur le moteur de classification N&B / Couleur (§ 5.2), qui reste la partie la plus incertaine du projet.
- Revue de sécurité à chaque étape sensible (authentification, paiement, stockage des fichiers), en s'appuyant sur la checklist du § 7.1.
- Rédaction de tests correspondant aux critères de recette du cahier des charges Phase 1 (§ 9).
- Pour la phase de codage proprement dite, Claude Code reste l'outil recommandé pour travailler directement dans le projet plutôt que de copier-coller du code entre une discussion et un éditeur.

Le rythme recommandé : un point de passage avec Claude à la fin de chaque semaine pour vérifier le jalon atteint avant de passer à la semaine suivante, plutôt qu'un seul échange en fin de projet.

7. Annexe — Checklists récapitulatives

7.1 Checklist sécurité (à vérifier avant le lancement, semaine 6)

- Accès opérateur/admin vérifié côté serveur (middleware), jamais uniquement côté client.
- Mots de passe hachés (bcrypt/argon2), jamais stockés en clair.
- Bucket de stockage des PDF privé, accès uniquement par liens signés à durée limitée.
- Règle de suppression automatique des fichiers en place et testée.
- Aucune donnée sensible dans les journaux d'application.
- Validation systématique côté serveur de tous les fichiers et formulaires, même déjà validés côté client.
- Requêtes base de données paramétrées (ORM), aucune requête SQL construite manuellement.
- Signature des webhooks de paiement vérifiée avant tout traitement.
- Aucune impression déclenchée sans confirmation serveur-à-serveur du paiement.
- HTTPS actif sur l'ensemble du site.
- Toutes les clés secrètes en variables d'environnement, absentes du dépôt Git.
- Limitation du débit (rate limiting) en place sur les routes sensibles.
- CORS restreint au seul domaine officiel du site.
- Dépendances du projet vérifiées (npm audit / pip-audit) sans faille critique connue.

7.2 Variables d'environnement et configuration à prévoir

Variable	Usage
DATABASE_URL	Chaîne de connexion à la base de données PostgreSQL (Neon).
R2_ACCESS_KEY_ID / R2_SECRET_ACCESS_KEY	Identifiants d'accès au bucket Cloudflare R2.
R2_BUCKET_NAME	Nom du bucket de stockage des fichiers PDF.

Variable	Usage
PAYMENT_API_KEY / PAYMENT_API_SECRET	Identifiants de l'agrégateur de paiement (CinetPay, PayDunya ou équivalent).
PAYMENT_WEBHOOK_SECRET	Secret utilisé pour vérifier la signature des webhooks de paiement.
JWT_SECRET	Clé secrète utilisée pour signer les jetons d'authentification de l'espace opérateur/admin.
WHATSAPP_API_KEY (ou équivalent)	Identifiants pour l'envoi des notifications client.